

Team 15 - Compiler Report 3

1. Summary of New Work

All changes extend the compiler from a pure IR-generation frontend into a complete compilation pipeline with optimisation and parallel executable output.

1.1 While Loop Support

A WhileNode AST node was added, following the same deferred-body pattern established for ForNode. The parser uses a body_stmt rule (introduced in the previous report) so body statements are never prematurely emitted.

1.2 Logical and Inequality Operators

Three new operator classes were added which were missed earlier to the lexer, parser, and AST:

- Inequality operator != (NEQ)
- Logical AND && (AND)
- Logical OR || (OR)
- Logical NOT ! (NOT)

Operator precedence is correctly set in the Bison grammar: || < && < comparison < arithmetic.

1.3 Function Definitions and Calls

Full function support was added across all compiler components:

- Lexer: func, return, void keywords added.
- Parser: func_def, func_call_stmt, return_stmt grammar rules. Function calls also work inside expressions (e.g. result = add(a, b);).
- AST: FunctionDefNode, FunctionCallNode, ReturnNode.
- Symbol table: FuncSignature struct stores return type and parameter types/names.

Functions are registered globally before the body is analysed, enabling recursive calls.

IR emitted for a function definition:

```
add [i32] {
  param x [i32]
  param y [i32]
  t1 = x + y [i32]
  return t1 [i32]
} // end add
IR emitted at a call site:
push_arg a [i32]
```

```
push_arg b [i32]
call add -> t2 [i32]
```

1.4 Scope-Stack Symbol Table

The flat `std::map` symbol table was replaced with a scope stack (`std::vector` of `std::unordered_map`). Each function body and block pushes a new scope on entry and pops it on exit. Variable lookup walks from innermost scope outward, supporting correct shadowing. Functions remain in a separate global table.

1.5 Optimiser with -O0 / -O1 / -O2 Flags

An optimisation pass (`optimizer.cpp`) was implemented, invoked by command-line flags matching real compiler conventions:

- -O0: no optimisation. Raw IR is output as generated.
- -O1: one round of constant folding + copy propagation + dead code elimination.
- -O2: same three passes repeated until no further changes occur (fixed-point iteration), then dead code elimination.

Pass descriptions:

- Constant folding: arithmetic and comparison instructions whose both operands are compile-time literals are evaluated at compile time. e.g. `t1 = 3 + 4` becomes `t1 = 7`.
- Copy propagation: assignments of the form `t = x` are tracked. Later uses of `t` are replaced by `x` directly, reducing unnecessary copies.
- Dead code elimination: temporaries that are assigned but never read are removed from the IR entirely.

1.6 OpenMP Code Generation Backend

A code generation pass (`codegen.cpp`) was implemented. It reads the IR vector and writes a compilable C file, then automatically invokes `gcc -O2 -fopenmp` to produce a real executable.

Key design decisions:

- Global variable declarations are sourced directly from the symbol table (`syntab.globalSymbols()`), ensuring every declared variable is emitted regardless of where it appears in the IR.
- Parallel loop detection: the codegen scans for the `PARALLEL LOOP` annotation injected by the dependency analyser. When found, it recognises the complete `init/label/bound/body/step/goto` pattern and emits a proper C for loop with `\#pragma omp parallel for schedule(static)` instead of a goto-based loop. This is required because OpenMP pragmas only work on structured C for loops.
- Sequential loops are emitted as goto-based C code matching the IR structure directly.
- Function definitions are emitted before `main()`, with forward declarations for call-order independence.

Usage:

```
./compiler -O2 --emit-c -o prog < input.bc
```

2. Features Demonstrated

Feature	Test Input	Status
While loop	<code>while (x != 0) { x = x - 1; }</code>	Working
Function definition	<code>func int32 add(int32 x, int32 y)</code>	Working
Function call (stmt)	<code>add(a, b);</code>	Working
Function call (expr)	<code>c = add(a, b);</code>	Working
-O0 flag	raw IR output	Working
-O1 flag	1 round optimisation	Working
-O2 flag	fixed-point optimisation	Working
--emit-c flag	generates C + compiles via gcc	Not Working
OpenMP parallel loop	<code>#pragma omp parallel for</code>	Not Working
Parallel executable	<code>./prog</code> runs on all CPU cores	Not Working

3. Deviations and Justifications

Backend: C + OpenMP instead of x86-64 assembly

The original plan proposed generating x86-64 assembly directly. This was revised:

- Direct assembly requires a full register allocator (graph colouring, NP-complete) and an instruction selector — 3,000–5,000 lines of infrastructure with no contribution to the parallelisation thesis.
- Generating C with OpenMP is how GCC, Clang, and Intel ICC implement their auto-parallelisation passes. The compiler's intellectual contribution — dependency analysis and parallelism classification — is fully preserved.
- The result is a real, runnable, multi-threaded executable, which is the required deliverable of a backend.

body_stmt rule introduced

A `body_stmt` grammar rule was added to separate statements inside loop/if bodies from top-level statements. This prevents premature IR emission - a bug where body statements called `generateIR()` during Bison reduction before the owning `ForNode` had set up loop context, producing ghost instructions before the loop in the generated C.

4. Planned Work for Next Demonstration

- Common Subexpression Elimination (CSE): detect that an expression computed multiple times in the same basic block can be computed once and the result reused.
- Loop-Invariant Code Motion (LICM): detect expressions inside a loop whose operands do not change across iterations and hoist them above the loop.
- Function inlining: replace calls to small functions with the function body directly, enabling further constant folding across call boundaries.
- CUDA kernel emission: for FULLY_PARALLEL loops where array size exceeds a threshold, emit a .cu kernel and invoke nvcc, distributing work across GPU threads automatically.
- While-loop dependency analysis: extend depcheck.cpp to classify while loops, currently conservatively treated as UNKNOWN.
- Fix OpenMP pragma placement for all parallel loop patterns, including nested parallel loops.